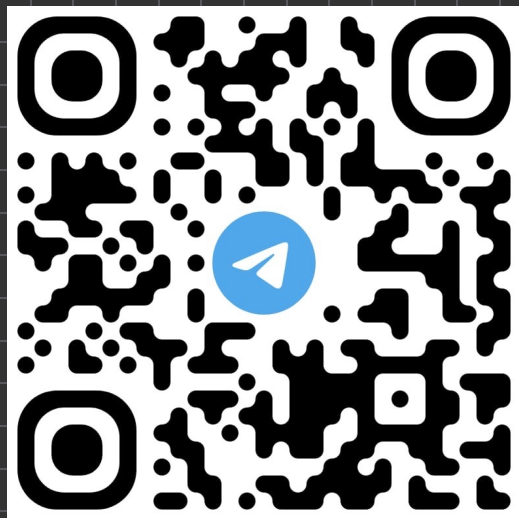


Coding and Web Development

Transition , Transform Animation

(Lecture 18)

Telegram



Discord



ek property ^{value} n/se dusri property value
mai smoothly change hona

→ User koi action krta hai

original state → changed state

CSS transition allows you to smoothly change property values over a specified duration rather than having the change happen instantly.

transition:

transition-duration
transition-property
transition-timing-function
transition-delay
transition-behavior

→ kitni der mai transition complete hoga

→ millisecond
ms
s → second

1s = 1000ms
1min = 60sec = 60 × 1000ms

The transition-duration property in CSS sets the exact length of time a transition animation takes to complete from start to finish. By default, this value is 0s, meaning any style changes will happen instantly unless you explicitly define a duration.

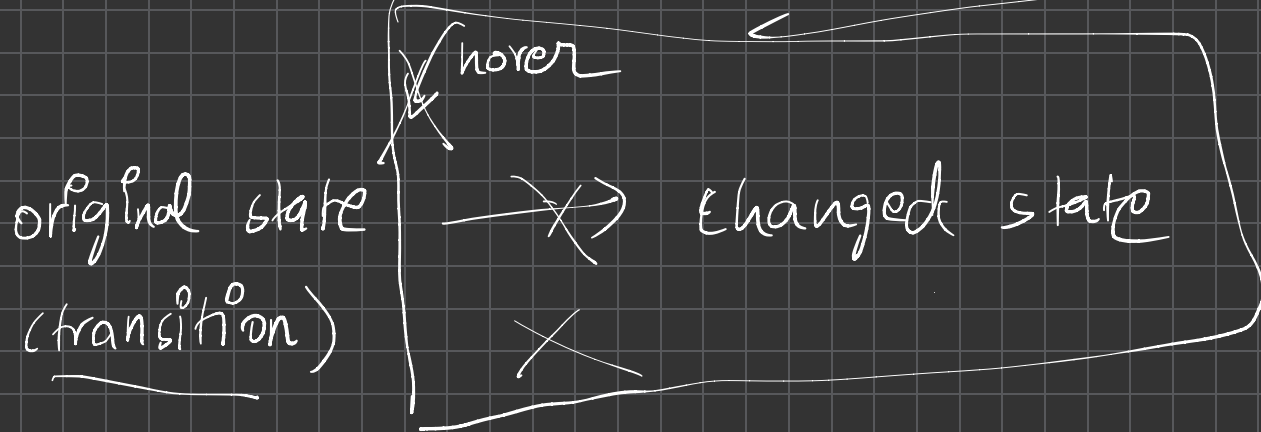
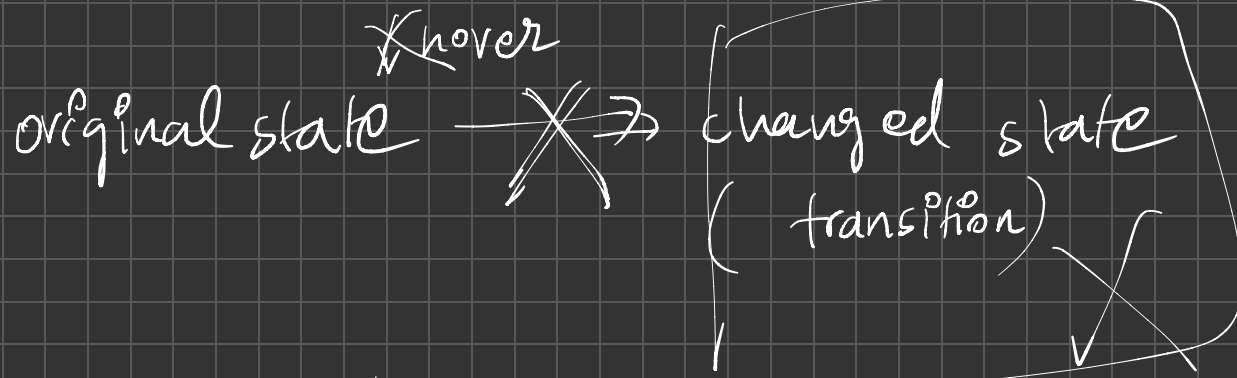
default → all

transition-duration
transition-property
transition-timing-function
transition-delay
transition-behavior

width, height, bgc

konisi CSS ki
property per
apply krna chate
ho

The transition-property CSS property specifies the names of the specific CSS properties that should undergo a smooth animation effect when their values change



user events

:hover, :focus, :active

- ✓ **transition-duration**
- ✓ **transition-property**
- transition-timing-function**
- transition-delay**
- transition-behavior**

speed of transition

linear → same
speed
for entire
transition



The transition-timing-function CSS property specifies the speed curve of a transition effect, allowing an element's property changes to vary in speed over its duration.

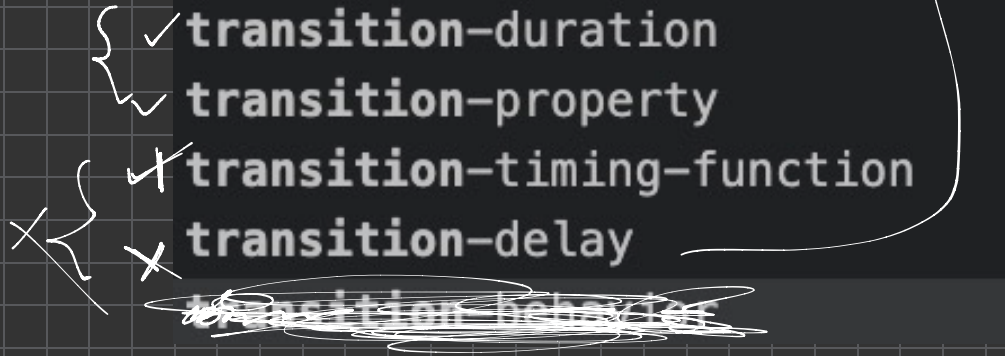
case-in

↳ initially speed team
badma? normal

case-out

↳ initially fast
badma? normal

kitni late transition start
hoga

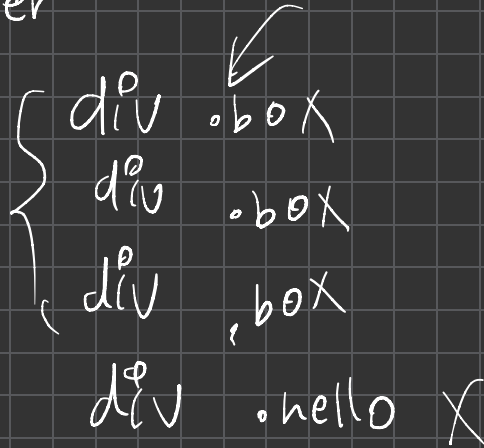


- ✓ **transition-duration**
- ✓ **transition-property**
- ✓ **transition-timing-function**
- X **transition-delay**
- X ~~**transition-behavior**~~

The transition-delay property in CSS specifies the amount of time to wait before starting a transition effect.

transition : property duration timing-function delay

.box:hover



- div .box
- div .box
- div .box
- div .hello X

• $\rightarrow$ div $\rightarrow$ parent

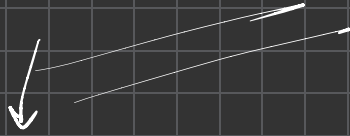
{ div
div
div

div

parent : horizon $\rightarrow$ div {

{
/
/
/
}

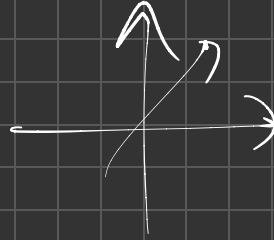
Transition → Transform



scale, move, rotate

The CSS transform property allows you to visually manipulate an element by rotating, scaling, moving (translating), or skewing it. It modifies the element's coordinate space without altering the normal document layout or pushing surrounding elements around.

translate → (x, y, z)



100% → 200%
1 → 2
x 1.5
150%

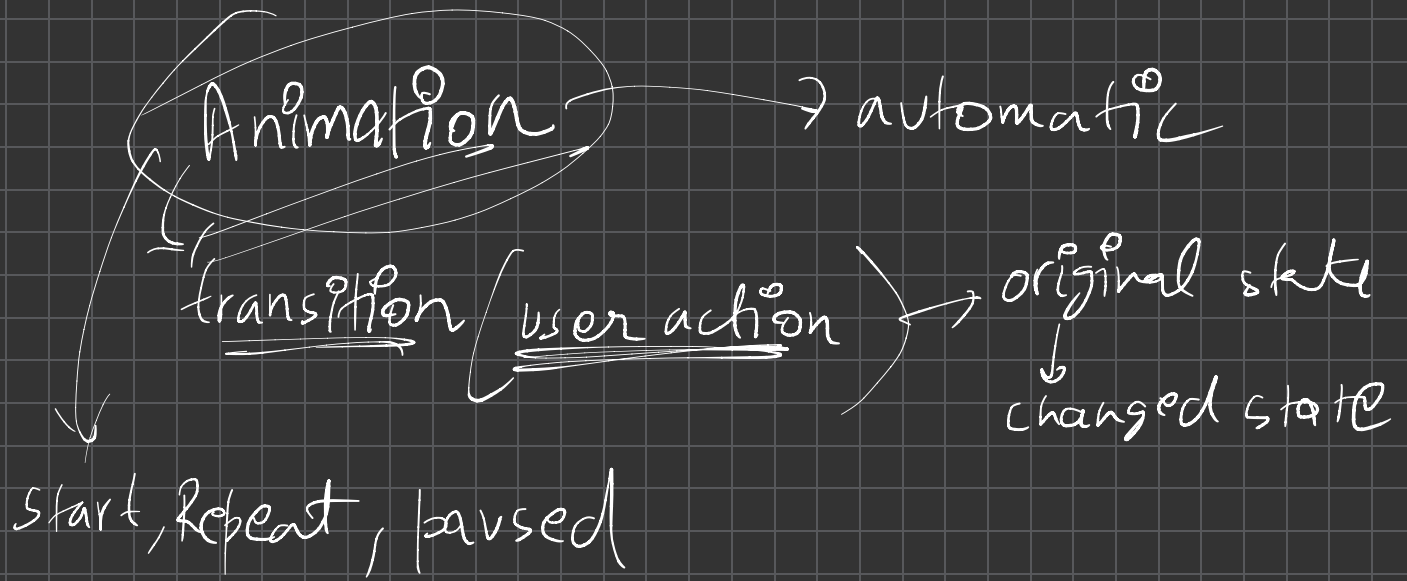
transform : translate(
px, rem, em
%) scale()

The `translate()` CSS function repositions an element in the horizontal and/or vertical directions.

The `scale()` CSS function defines a transformation that resizes an element on the 2D plane.

The `rotate()` CSS function defines a transformation that rotates an element around a fixed point on the 2D plane, without deforming it.

rotate (
 ↗
 deg



CSS animations allow you to transition elements smoothly from one style configuration to another without using heavy JavaScript. They rely on two fundamental building blocks: the **@keyframes** rule (defining the intermediate steps) and the animation sub-properties (binding the rules to elements)

```
animation: name duration timing-function delay iteration-count direction fill-mode;
```

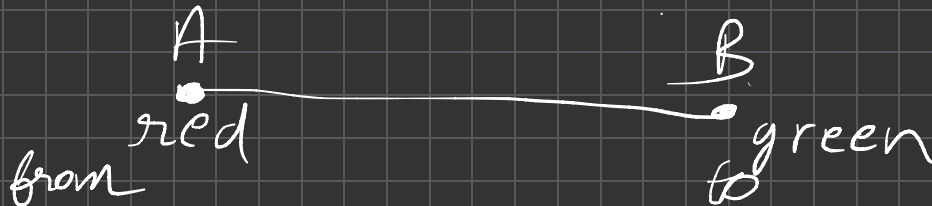
animation
name

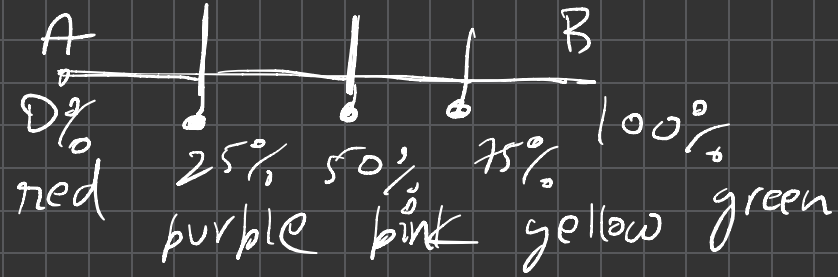
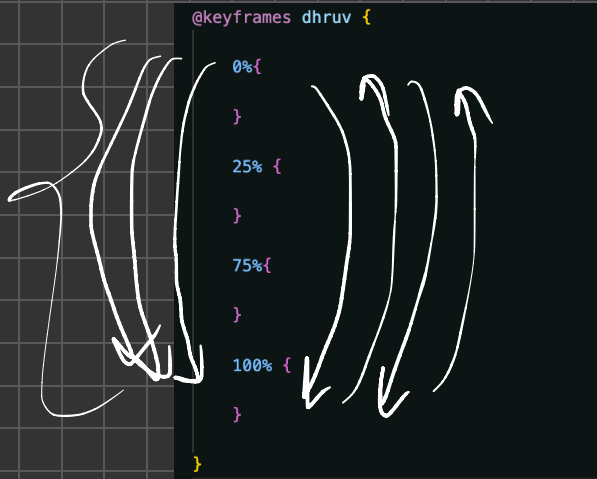
keyword
↗

```
@keyframes sneha {  
  ≡ steps  
}
```

@keyframes
↓
steps

```
> @keyframes sneha {  
  > from {  
    } → initial  
  > to {  
    } → final  
    } → code  
}
```





@keyframes is like a storyboard or a timeline for a CSS animation. It lets you decide exactly what a web element should look like at specific checkpoints from the moment the animation starts until it finishes.

`animation: name duration timing-function delay iteration-count direction fill-mode;`

↓
kitna time
ho

↑ ↑ ↑
↓
kitni bar
animation
karana chate
ho
1, 2, 3, 100,

The animation-iteration-count CSS property defines the number of times an animation sequence plays before stopping.

Syntax

code dikhane ka tarika

```
.className {  
    →  
    →  
    →  
}
```

```
@keyframes animation_name {
```

```
    from {
```

```
        0% {
```

```
    to {
```

```
        50% {
```

```
        100% {
```

```
}
```

animation
→ kaha par
stop hoga

```
animation: name duration timing-function delay iteration-count direction fill-mode;
```

animation kis direction se
start ho
→ reverse
→ normal
→ alternate
→ alternate-reverse

The animation-direction CSS property determines whether an animation plays forward, backward, or alternates back and forth between the two. It allows you to reuse a single @keyframes timeline in multiple ways without rewriting your code.

The CSS animation-fill-mode property specifies how an animation applies styles to its target before it starts (during a delay) and after it finishes executing.

